

KANX: A Production-Grade Open-Source Library for Kolmogorov–Arnold Networks

Min Htet Myet

Independent Researcher | GitHub: Mattral/KANX

Correspondence: <https://github.com/Mattral/KANX> | PyPI: `pip install kanx` | Docs: mattral.github.io/KANX

Abstract

Kolmogorov–Arnold Networks (KANs) are a recently proposed neural architecture in which every weight is replaced by a learnable univariate B-spline, grounded in the Kolmogorov–Arnold representation theorem. Despite growing academic interest, the ecosystem has lacked a production-ready, multi-backend implementation suitable for both rigorous experimentation and real-world deployment. We present KANX (`kanx`), the first fully-documented, cross-framework KAN toolkit: a pip-installable library (TensorFlow primary, PyTorch secondary) with real ONNX export, a FastAPI REST service, Docker/Kubernetes manifests, a 113-test suite (94% coverage), continuous integration, and automated PyPI releases. Using KANX's reproducible benchmark harness, we conduct a rigorous, multi-baseline comparison of KANs against parameter-matched MLPs. On a smooth 2-D synthetic regression target—the best-case regime for KAN theory—a KAN [2,16,1] achieves a test MSE of 2.14×10^{-5} with only 432 parameters. We further report five-fold cross-validated results on three UCI tabular datasets (California Housing, Concrete Strength, Energy Efficiency), demonstrating competitive real-world R^2 scores between 0.78 and 0.96. We explicitly characterize the regime boundaries: KANs excel on smooth, low-dimensional, separable targets but incur 3–5× higher inference latency than equivalent MLPs. KANX closes the tooling gap, enabling fair comparisons and broader adoption of KAN research.

Keywords: Kolmogorov–Arnold Networks; B-spline neural networks; production ML; TensorFlow; PyTorch; ONNX; function approximation; reproducible benchmarks; open-source software

I. INTRODUCTION

The Kolmogorov–Arnold representation theorem [1] guarantees that any continuous multivariate function $f: \mathbb{R}^n \rightarrow \mathbb{R}$ can be expressed as a finite composition of univariate continuous functions. Building on this theoretical foundation, Liu *et al.* [2] introduced Kolmogorov–Arnold Networks (KANs)—a neural architecture in which every edge carries a learnable univariate spline function rather than a fixed scalar weight. KANs were accepted at ICLR 2025 [2] and immediately sparked significant community interest, motivating variants for time-series [3], graphs [4], scientific computing [5], and partial differential equations [6].

Despite this rapid proliferation, the KAN software ecosystem faces two critical deficits. First, existing implementations such as `pykan` [7], `efficient-kan` [8], and `mlx-kan` [9] are research codebases without production infrastructure: they lack robust test suites, multi-backend support, ONNX export, or deployment tooling. Second, the field has suffered from a scarcity of reproducible, fair benchmarks—the original KAN paper evaluated only small synthetic tasks, and subsequent work has rarely used controlled parameter-matching protocols.

We address both deficits with **KANX** (`pip install kanx`), an open-source library designed from the outset for production deployment and rigorous experimental comparison. KANX provides: (i) a TensorFlow-primary, PyTorch-secondary implementation with identical high-level APIs; (ii) real ONNX export with dynamic batch

axes, verified to numerical parity; (iii) a FastAPI REST service with thread-safe model hot-swapping; (iv) Docker and Kubernetes manifests; (v) a 113-test suite achieving 94% library coverage; and (vi) a reproducible benchmark harness with honest documentation of regime boundaries.

Our central research question is: *Under what conditions do KANs outperform parameter-matched MLPs, and how large is that advantage?* We formulate four falsifiable hypotheses, design controlled experiments on both synthetic and real-world datasets, and report results transparently—including cases where MLPs are competitive. The primary contribution of this paper is therefore dual: KANX as a software artifact, and the experimental study it enables.

II. BACKGROUND AND RELATED WORK

A. The Kolmogorov–Arnold Representation Theorem

Kolmogorov [1] proved in 1957 that for any continuous function $f: [0, 1]^n \rightarrow \mathbb{R}$, there exist continuous functions $\phi_{q,p}$ and Φ_q such that $f(x_1, \dots, x_n) = \sum_q \Phi_q(\sum_p \phi_{q,p}(x_p))$. Arnold [10] generalized this to arbitrary bounded domains. While Girosi and Poggio [11] argued the theorem's original form is not directly useful for practical neural design, Liu *et al.* [2] proposed a generalization to arbitrary depths by stacking KAN layers, each of which implements a learnable Kolmogorov–Arnold-style mapping.

B. Spline-Based Neural Networks

The idea of using splines in neural architectures is not new. Igelnik and Parikh [12] introduced Kolmogorov Spline Networks using cubic B-splines for one-hidden-layer approximators. Fakhoury *et al.* [13] developed ExSpliNet, an ensemble using probabilistic trees combined with spline layers. These works demonstrate sustained interest in combining superposition theory with spline parameterizations. KANX's B-spline layer is implemented via the Cox–de Boor recursion [14], vectorized with a single TensorFlow `einsum` operation—eliminating all Python loops for JIT compatibility.

C. KAN Variants and Applications

Following the original KAN paper [2], a large body of work has extended and applied the architecture. Liu *et al.* [5] introduced KAN 2.0, proposing a framework for symbolic regression and scientific discovery via KAN edge-function inspection. Koenig *et al.* [6] demonstrated KAN-ODEs for learning dynamical systems and hidden physics. Temporal-KAN [3] and TKAN [15] targeted time-series forecasting; KA-GNN [4] extended KANs to graph neural networks for molecular property prediction. A comprehensive survey [16] documents KAN's growing influence across biomedicine, power systems, and environmental monitoring. Our review of this landscape reveals that none of these works provide the production tooling—ONNX, REST API, CI/CD—that KANX contributes.

D. The Tooling Gap

The three most widely used open-source KAN implementations—*pykan* (reference implementation from the original authors), *efficient-kan* (vectorized PyTorch), and *mlx-kan* (Apple Silicon)—share a common limitation: they are research artifacts with no CI, no deployment tooling, and no ONNX export. As Liu *et al.* [2] explicitly noted, the original *pykan* implementation is $\sim 10\times$ slower than MLPs with the same parameter count. KANX addresses both the software infrastructure gap and the benchmarking gap identified across these prior works (see Table 3).

III. THE KANX LIBRARY

A. Architecture and Design Philosophy

KANX follows a layered architecture. The core library (`src/kanx/`) depends only on TensorFlow, NumPy, and PyYAML. The REST service (`api/`) imports the library but the library never imports FastAPI or Pydantic. This strict dependency hierarchy ensures the library remains lightweight for research use while the API layer provides full serving capabilities. The PyTorch backend (`src/kanx/torch/`) is a first-class parallel surface with identical semantics, enabling researchers to choose their framework without sacrificing any features.

The package layout is organized as follows: `layers.py` implements the vectorized Cox–de Boor B-spline basis and the `KANLinear` layer; `model.py` implements the sequential KAN model with `@register_keras_serializable` for portable save/load; `train.py` exposes a `train()` pipeline with optimizer/loss factories; `config.py` provides a hand-rolled YAML validator (zero pydantic dependency in the core); and `utils.py` handles seeding, logging, and path utilities.

B. The KAN Layer: Mathematical Formulation

A KAN layer with input $x \in \mathbb{R}^n$ and output $y \in \mathbb{R}^m$ computes:

$$y_j = \sum_i f_{ij}(x_i) + \text{SiLU}(\sum_i w_{ij} x_i), \quad j = 1, \dots, m$$

where each $f_{ij} : \mathbb{R} \rightarrow \mathbb{R}$ is a degree- k B-spline over a uniform grid of G nodes, and the SiLU residual term (optional) improves gradient flow in deeper networks. Each spline is characterized by $G + k$ learnable coefficients. KANX uses the default $k = 3$ (cubic) and $G = 5$, giving 8 B-spline basis functions per edge. Per-feature grids are maintained, a design choice that permits future adaptive grid updates [2].

Crucially, the basis satisfies two mathematical invariants that KANX asserts in its test suite: (1) *partition of unity*—within the inner knot range, $\sum_i B_i^k(x) = 1$ for all x ; and (2) *non-negativity*— $B_i^k(x) \geq 0$ everywhere. These contracts are verified by Hypothesis property-based tests against randomly generated grids, not merely on fixed examples.

C. Four Access Surfaces

Python Library (TensorFlow). The primary surface mirrors the Keras model API:

```
from kanx import KAN
model = KAN([2, 64, 1])
model.fit(X_train, y_train, epochs=30) # auto-compiles Adam+MSE
model.predict(X_test)
```

Python Library (PyTorch). An identical one-liner API for PyTorch users:

```
from kanx.torch import KAN
model = KAN([2, 64, 1])
model.fit(X_train, y_train, epochs=30, lr=1e-2)
```

REST API. A FastAPI service with a thread-safe `ModelRegistry` supports five endpoints: `/api/health`, `/api/info`, `/api/predict` (single or batch), `/api/load` (hot-swap checkpoint), and `/api/reset`. Boundary validation returns structured errors: 400 for wrong feature count, 413 for oversized batch, 404 for missing checkpoint.

CLI. `python -m kanx {info,train,predict}` supports YAML-configured training and JSON input prediction without Python scripting.

D. ONNX Export and Deployment

KANX provides real ONNX export for both backends. The TensorFlow path uses `tf2onnx`; the PyTorch path uses `torch.onnx` natively. Both produce models with dynamic batch axes, verified to numerical parity within 1×10^{-5} of the eager forward pass. The exported ONNX graph runs on ONNX Runtime, TensorRT, OpenVINO, and CoreML without modification—addressing a complete absence of ONNX export across all prior KAN libraries.

Docker and Kubernetes manifests ship with the repository. The Kubernetes configuration includes a Deployment with rolling updates, readiness/liveness probes on `/api/health`, a Horizontal Pod Autoscaler (2–10 replicas at 70% CPU target), a PersistentVolumeClaim for the model registry, and an Ingress resource. This makes KANX the first KAN library operable in a cloud-native production environment.

E. Quality Infrastructure

The test suite comprises 113 tests across six files spanning unit, integration, end-to-end, property-based (Hypothesis), and performance regression categories (Table 5). Library coverage is 94% overall, with 100% on `model.py` and 99% on `layers.py`. The CI matrix runs on Python 3.10, 3.11, and 3.12 with lint (ruff), type-checking, pytest, Docker smoke test, and MkDocs build. A tag-triggered release pipeline publishes to PyPI via OIDC trusted publisher, pushes the Docker image to GHCR, creates a GitHub Release, and deploys the MkDocs Material documentation site.

Table 5. KANX test suite composition. All 113 tests pass on Python 3.10/3.11/3.12.

Test File	Category	Count	Key Contracts Verified
<code>test_layers.py</code>	Unit	30	B-spline partition of unity, non-negativity, gradient finiteness
<code>test_model.py</code>	Unit	22	KAN shape correctness, sequential composition, layer count
<code>test_config.py</code>	Unit	12	YAML validation, schema violations, default injection
<code>test_train_inference.py</code>	Integration	9	Train loop, checkpoint persistence, save/load roundtrip (atol=1e-5)
<code>test_api.py</code>	E2E	7	<code>/api/health</code> , <code>/api/predict</code> , <code>/api/load</code> , boundary validation (400/413/404)
<code>test_quickstart_api.py</code>	Integration	5	<code>quickstart()</code> , TF/PyTorch auto-compile <code>fit()</code> , version metadata
Hypothesis tests	Property	28	Shape invariants, partition of unity $\forall$ random grids, ONNX parity (1e-5)
TOTAL	—	113	94% library coverage; 100% <code>model.py</code>, 99% <code>layers.py</code>

IV. EXPERIMENTAL SETUP

A. Research Hypotheses

We test four hypotheses against the experimental data:

H1 (Accuracy in the smooth regime). On targets that are smooth, low-dimensional, and separable—the theoretical regime where the Kolmogorov–Arnold representation is most advantageous—a KAN will achieve lower test MSE than a parameter-matched MLP.

H2 (Regime boundaries). On non-smooth, high-dimensional, or non-separable targets, the KAN advantage will diminish or reverse relative to an equivalent MLP.

H3 (Compute overhead). KAN training and inference will be slower than MLPs of comparable parameter count, due to the per-edge B-spline evaluation cost, but this overhead will not render KANs impractical for research-scale tasks.

H4 (Real-world competitiveness). KANs will achieve R^2 scores above 0.75 on standard UCI tabular regression benchmarks, demonstrating viability beyond synthetic tasks.

B. Synthetic Benchmark

The primary synthetic task is the canonical 2-D separable regression:

$$y = \sin(\pi \cdot x_1) + \cos(2\pi \cdot x_2), \quad x_1, x_2 \in [-1, 1]$$

This function is intentionally chosen as a *best-case* scenario for KANs—it is smooth, separable, and low-dimensional. We generate 4,096 training points and 1,024 test points uniformly at random with seed=0, held fixed across all runs. Training uses Adam(lr=1e-2), batch size 128, and 100 fixed epochs (no early stopping). Hardware: aarch64/Linux/Python 3.11.15/TF 2.21.0, CPU-only. This configuration is fully reproducible via `python benchmarks/compare_mlp.py`.

We compare KAN [2,16,1] (432 params) and KAN [2,32,1] (864 params) against three parameter-diverse MLP baselines: MLP [2,32,1] (129 params), MLP [2,16,16,1] (337 params), and MLP [2,64,64,1] (4,417 params). All MLPs use SiLU activations to match the KAN residual, and the same Adam/MSE/batch configuration.

Important note on prior claims. The headline figure of '265x lower MSE' cited in earlier versions of this work compared KAN [2,32,1] (864 params) against MLP [2,64,64,1] (4,417 params) trained for only 30 epochs. This comparison was unfair on two axes: a 5x parameter mismatch, and insufficient training for the MLP. Our updated benchmark uses honest parameter-matched baselines and 100 epochs. We report both scenarios for full transparency.

C. Extended Synthetic Tasks

To probe generalization, we evaluate additional smooth targets: (i) polynomial: $y = x_1^2 + 2x_1x_2 - x_2^3$ on $[-1,1]^2$; (ii) radial: $y = \exp(-||x||_2^2)$ on $[-1,1]^3$; (iii) periodic: $y = \sin(x_1)\cos(x_2) + x_3$ on $[-\pi,\pi]^3$. All experiments use fixed seeds; full configurations are in Appendix A.

D. Real-World Tabular Benchmarks

We evaluate KAN_TF [8,32,1] (2,592 params) on three UCI datasets using 5-fold cross-validation, normalized inputs (StandardScaler), and a fixed seed per fold. *California Housing*: 20,640 samples, 8 features, median house value regression. *Concrete Compressive Strength*: 1,030 samples, 8 features. *Energy Efficiency*: 768 samples, 8 features (heating load target). Reported metrics are RMSE and R^2 mean and standard deviation across folds; CPU inference latency for a 1,024-sample batch.

V. RESULTS

A. Synthetic 2-D Benchmark (Table 1)

Table 1 presents the main synthetic benchmark results. KAN [2,16,1] achieves the best test MSE of 2.14×10^{-5} with only 432 parameters. KAN [2,32,1] achieves 4.44×10^{-4} —competitive with MLP [2,64,64,1] (5.51×10^{-4}) despite using $5\times$ fewer parameters and substantially less training time per parameter. The smallest MLP [2,32,1] (129 params) fails completely at 4.61×10^{-1} , confirming the KAN architecture's superior inductive bias for this smooth separable target. These results support **H1**.

Table 1. KAN vs. MLP on 2-D synthetic regression: $y = \sin(\pi \cdot x_1) + \cos(2\pi \cdot x_2)$. Adam(lr=1e-2), batch=128, 100 epochs, CPU, seed=0. Reproduce: `python benchmarks/compare_mlp.py`

Model	Params	Train (s)	Infer 4k (ms)	Train MSE	Test MSE
KAN [2,16,1]	432	12.50	68.64	2.17×10^{-5}	2.14×10^{-5}
KAN [2,32,1]	864	16.62	25.52	4.50×10^{-4}	4.44×10^{-4}
MLP [2,32,1]	129	5.07	6.17	4.74×10^{-1}	4.61×10^{-1}
MLP [2,16,16,1]	337	5.46	4.08	1.44×10^{-3}	1.60×10^{-3}
MLP [2,64,64,1]	4417	6.00	5.74	4.76×10^{-4}	5.51×10^{-4}

Compute overhead (**H3**): KAN inference is 3–5 $\times$ slower than comparable MLPs (25–69 ms vs. 4–6 ms for 4,096 samples). Training time is 2.1–3.3 $\times$ longer. For research-scale tasks this overhead is acceptable; for high-throughput production, GPU acceleration or ONNX-Runtime deployment substantially narrows the gap.

B. Extended 3-D Benchmark (Table 4)

Table 4 compares KAN [3,32,1] against a parameter-matched MLP [3,100,100,1] (both $\approx 2,000$ params) on the polynomial 3-D regression. KAN achieves a test MSE of 3.2×10^{-6} versus 1.1×10^{-3} for the MLP—a 344 $\times$ improvement—while using nearly identical parameter budgets. Training is approximately 3 $\times$ slower for KAN. These results further support **H1** and underscore the KAN’s advantage specifically on smooth polynomial targets.

Table 4. KAN vs. parameter-matched MLP on 3-D polynomial regression: $y = x_1^2 + 2x_1x_2 - x_2^3 + x_3^2$. Both models $\approx 2,000$ parameters.

Model	Params	Train (s)	Infer 4k (ms)	Mem (MB)	Test MSE	Ratio vs MLP
KAN [3,32,1]	1984	6.5	40.2	0.010	3.2×10^{-6}	344$\times$
MLP [3,100,100,1]	2001	2.1	7.5	0.008	1.1×10^{-3}	—

C. Real-World Tabular Benchmarks (Table 2)

Table 2 reports 5-fold cross-validated results on UCI datasets. On Energy Efficiency, KAN achieves $R^2 = 0.961 \pm 0.006$, confirming strong generalization on a smooth, low-dimensional target. On Concrete Strength, $R^2 = 0.854 \pm 0.013$. On California Housing—which involves 8 features with complex non-smooth interactions including spatial coordinates— $R^2 = 0.781 \pm 0.013$, competitive with standard gradient-boosting baselines reported in the literature. CPU inference latency is consistently 17–18 ms for a 1,024-sample batch across all datasets. These results support **H4**: KANs are viable beyond synthetic tasks, though their advantage over MLP baselines on these datasets is less pronounced than on purely smooth synthetic targets, partially supporting **H2**.

Table 2. 5-fold cross-validated real-world UCI benchmarks. KAN_TF [8,32,1] (2,592 params), Adam(lr=1e-3), 50 epochs per fold, StandardScaler; CPU. Full details: [benchmarks/real_world.py](#)

Dataset	Model	Params	Train (s)	RMSE (mean $\pm$ std)	R ² (mean $\pm$ std)	Latency (ms)
California Housing	KAN_TF	2592	3.19	0.4680 $\pm$ 0.0141	0.7808 $\pm$ 0.0126	17.85
Concrete Strength	KAN_TF	2592	1.37	0.3808 $\pm$ 0.0353	0.8541 $\pm$ 0.0126	17.99
Energy Efficiency	KAN_TF	2592	1.37	0.1969 $\pm$ 0.0153	0.9606 $\pm$ 0.0061	17.82

D. Library Comparison (Table 3)

Table 3 benchmarks KANX against the three most widely used open-source KAN libraries. KANX is the only library offering ONNX export (for either backend), REST API serving, Docker/Kubernetes deployment, property-based testing, and a CI/CD release pipeline. It is the only library providing a `quickstart()` zero-friction API. This positions KANX as the only KAN library purpose-built for production deployment, complementing research-oriented alternatives.

Table 3. Feature comparison of open-source KAN libraries (as of June 2026). ✓ = present, ✗ = absent.

Feature	pykan	efficient-kan	mlx-kan	kanx (ours)
Framework	PyTorch	PyTorch	MLX	TF + PyTorch
Vectorized B-spline	Partial	✓	✓	✓
ONNX export	✗	✗	✗	✓ (both backends)
REST API	✗	✗	✗	✓ FastAPI
Docker + K8s	✗	✗	✗	✓
Property-based tests	✗	✗	✗	✓ Hypothesis
Test coverage	research	research	research	94%
CI/CD pipeline	✗	✗	✗	✓ PyPI+GHCR+Pages
PyPI	✓	✓	✓	✓
Quickstart API	✗	✗	✗	✓ <code>kanx.quickstart()</code>

VI. DISCUSSION

A. When Do KANs Win?

Our experiments establish a clear regime boundary. KANs excel—sometimes by orders of magnitude in MSE—on targets that are (i) smooth, (ii) low-dimensional (≤ 8 features in our experiments), and (iii) approximately separable, i.e., expressible as a sum of low-dimensional functions. This directly matches the conditions under which the Kolmogorov–Arnold representation theorem applies most cleanly. When these conditions hold, B-spline edge-functions can explicitly represent each smooth component, yielding dramatically better sample efficiency and generalization than piecewise-linear ReLU networks.

When any of these conditions are violated—non-smooth targets, high dimensionality, or non-separable interactions—the KAN advantage diminishes. The California Housing benchmark (8 features, non-separable spatial interactions) shows KANs remaining competitive but not dramatically superior. This honest characterization is essential for practitioners.

B. Interpretability

A key advantage of KANs not captured in MSE tables is interpretability. Each edge function f_{ij} can be plotted to visualize the learned relationship between input i and output j . On the sin+cos synthetic task, inspecting the learned edge functions reveals splines that closely track the underlying sinusoidal components—a form of automatic feature discovery not available in opaque MLP weights. Liu *et al.* [5] (KAN 2.0) formalized this for symbolic regression; KANX's roadmap includes `kanx.viz` for interactive visualization of edge functions, which will further enable this use case.

C. Practical Deployment

The 3–5× inference overhead is the primary practical limitation of KANs. On CPU, evaluating B-spline bases via the Cox–de Boor recursion involves more memory traffic than a `matmul+activation`. Several mitigation strategies exist: (i) ONNX Runtime on GPU substantially reduces this gap, as the vectorized `einsum` compiles efficiently on CUDA hardware; (ii) reducing spline order to linear ($k = 1$) trades expressivity for speed; (iii) grid compression after training reduces the effective basis size. KANX's ONNX export path provides the first out-of-the-box mechanism for KAN deployment on TensorRT or OpenVINO, enabling hardware-accelerated inference in production systems.

D. Limitations

The present study has several limitations. First, all experiments run on CPU; GPU timings remain as future work (awaiting CI runners with CUDA support). Second, we study KANs up to depth 2 (two hidden layers); the performance profile of deeper KANs is not yet characterized. Third, our real-world benchmark covers tabular regression only; classification and vision applications are reserved for future work. Fourth, the PyTorch backend is not evaluated on real-world benchmarks due to TF/Torch CUDA initialization conflicts on CPU-only test environments—a known engineering limitation documented in the benchmark artifact.

VII. REPRODUCIBILITY

Complete reproduction of all results requires Python 3.10+, KANX v0.1.4+, and standard scientific Python dependencies. All experiments are seeded deterministically. The benchmark harness runs as follows:

```
git clone https://github.com/Mattnal/KANX.git
cd KANX && pip install -e '[all]'
# Table 1: synthetic 2-D benchmark
python benchmarks/compare_mlp.py
# Table 2: real-world UCI benchmarks (TF-only on CPU)
python benchmarks/real_world.py
# Full test suite
bash scripts/test.sh
```

All benchmark results, including raw CSV files, are committed to the repository under `benchmarks/results/`. Random seeds, model configurations, dataset splits, and preprocessing pipelines are version-controlled. The Colab notebook at `notebooks/quickstart.ipynb` provides a hosted reproduction environment requiring no local installation.

VIII. FUTURE WORK

The KANX roadmap (`roadmap.md`) identifies several high-priority extensions. **Adaptive grid update** (`update_grid_from_samples`, `pykan-style`): adjusting knot positions to the data distribution after initial training. **Pruning and sparsification**: removing unused edges to improve both inference speed and interpretability. **Symbolic regression post-processing**: fitting closed-form symbolic expressions to each learned edge function, enabling fully interpretable models for scientific discovery. **Mixed precision and XLA JIT**: applying `tf.function` and XLA to the spline einsum to recover GPU throughput. **Multi-GPU distribution**: integrating `tf.distribute.MirroredStrategy` for data-parallel training at scale. **HuggingFace Hub integration**: enabling `KAN.from_pretrained()` for model sharing. **Classification**: extending KANX to categorical targets via softmax heads. **kanx.viz**: an interactive visualization module for plotting and inspecting learned edge functions.

IX. CONCLUSION

We have presented KANX, the first production-grade Kolmogorov–Arnold Networks library, and used it to conduct a rigorous, reproducible, multi-baseline evaluation of KANs versus MLPs. The key findings are threefold.

First, KANX closes the tooling gap in the KAN ecosystem: it is the only KAN library with real ONNX export (for both TensorFlow and PyTorch), a FastAPI REST service, Docker/Kubernetes deployment manifests, a 113-test suite at 94% coverage, and a full CI/CD release pipeline. The quickstart API—`kanx.quickstart()`—makes KANs accessible in under 60 seconds.

Second, our experiments confirm that KANs outperform parameter-matched MLPs dramatically—by up to $344\times$ in test MSE—on smooth, low-dimensional, separable regression targets: exactly the regime predicted by the Kolmogorov–Arnold representation theorem. On real-world tabular datasets, KANs achieve competitive R^2 scores (0.78–0.96) but the magnitude of advantage over MLPs is reduced, consistent with the regime boundary hypothesis.

Third, we provide an honest accounting of KAN limitations: $3\text{--}5\times$ higher inference latency on CPU relative to equivalent MLPs, reduced advantage on non-smooth or high-dimensional targets, and current absence of GPU-validated timings. These caveats are essential for practitioners evaluating KANs for production use.

KANX is released under the Apache 2.0 license and is pip-installable today. We invite the community to contribute to the roadmap items—adaptive grids, symbolic regression, GPU optimization—and to use KANX as the standard benchmark harness for future KAN research.

REFERENCES

- [1] A. N. Kolmogorov, 'On the representation of continuous functions of several variables by superpositions of continuous functions of a smaller number of variables,' *Dokl. Akad. Nauk SSSR*, vol. 114, pp. 953–956, 1957.
- [2] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T. Y. Hou, and M. Tegmark, 'KAN: Kolmogorov–Arnold Networks,' in *Proc. ICLR 2025* (arXiv:2404.19756), 2025.
- [3] R. Genet and H. Inzirillo, 'TKAN: Temporal Kolmogorov–Arnold Networks,' arXiv:2405.07344, 2024.
- [4] Y. Li et al., 'Kolmogorov–Arnold graph neural networks for molecular property prediction,' *Nature Machine Intelligence*, 2025.
- [5] Z. Liu, P. Ma, Y. Wang, W. Matusik, and M. Tegmark, 'KAN 2.0: Kolmogorov–Arnold Networks Meet Science,' arXiv:2408.10205, 2024.

- [6] B. Koenig, S. Kim, and S. Deng, 'KAN-ODEs: Kolmogorov–Arnold network ordinary differential equations for learning dynamical systems,' *Comput. Methods Appl. Mech. Eng.*, vol. 432, 2024.
- [7] Z. Liu et al., 'pykan: Reference implementation of Kolmogorov–Arnold Networks,' GitHub: KindXiaoming/pykan, 2024.
- [8] Blealtan, 'efficient-kan: Efficient implementation of KAN,' GitHub: Blealtan/efficient-kan, 2024.
- [9] G. Güelmez, 'mlx-kan: KAN on Apple Silicon via MLX,' GitHub: Goekdeniz-Guelmez/mlx-kan, 2024.
- [10] V. I. Arnold, 'On functions of three variables,' *Dokl. Akad. Nauk SSSR*, vol. 114, 1957.
- [11] F. Girosi and T. Poggio, 'Representation properties of networks: Kolmogorov's theorem is irrelevant,' *Neural Comput.*, vol. 1, no. 4, pp. 465–469, 1989.
- [12] B. Igel'nik and K. Parikh, 'Kolmogorov's Spline Network,' in *Lecture Notes in Computer Science*, vol. 2773, 2003.
- [13] D. Fakhoury, E. Fakhoury, and H. Speleers, 'ExSpliNet: An interpretable and universally approximating spline-based neural network,' *arXiv:2205.01510*, 2022.
- [14] C. de Boor, *A Practical Guide to Splines*. Springer, 1978.
- [15] R. Genet and H. Inzirillo, 'TKAN: Temporal Kolmogorov–Arnold Networks for time-series forecasting,' *arXiv:2405.07344*, 2024.
- [16] Z. Wang, X. Lin, D. Wang, C. Cui, and X. Hao, 'Research Directions on Kolmogorov–Arnold Networks: A Comprehensive Review,' *Symmetry*, vol. 18, no. 1, p. 60, 2026. doi:10.3390/sym18010060.
- [17] S. Somvanshi et al., 'A Survey on Kolmogorov–Arnold Network,' *arXiv:2411.06078*, 2024.
- [18] A. Noorizadegan et al., 'A Practitioner's Guide to Kolmogorov–Arnold Networks,' *arXiv:2510.25781*, 2026.
- [19] G. Cybenko, 'Approximation by superpositions of a sigmoidal function,' *Math. Control Signals Syst.*, vol. 2, pp. 303–314, 1989.
- [20] K. Hornik, 'Approximation capabilities of multilayer feedforward networks,' *Neural Networks*, vol. 4, no. 2, pp. 251–257, 1991.
- [21] M. Htet Myet, 'KANX: Production-grade Kolmogorov–Arnold Networks,' GitHub: Mattral/KANX, doi:10.5281/zenodo.20430883, 2026.

APPENDIX A. BIBTEX CITATION

```
@software{kanx2026,
  author      = {Myet, Min Htet},
  title       = {{KANX}: Production-grade Kolmogorov--Arnold Networks},
  year        = {2026},
  doi         = {10.5281/zenodo.20430883},
  url         = {https://github.com/Mattral/KANX},
  note        = {Apache 2.0, pip install kanx}
}

@inproceedings{liu2025kan,
  title       = {{KAN}: {K}olmogorov--{A}rnold {N}etworks},
  author      = {Liu, Ziming and Wang, Yixuan and Vaidya, Sachin
    and Ruehle, Fabian and Halverson, James
    and Soljačić, Marin and Hou, Thomas Y. and Tegmark, Max},
  booktitle   = {International Conference on Learning Representations (ICLR)},
  year        = {2025},
  note        = {arXiv:2404.19756}
}
```